

PLaND Path to Least Non Determinism

Maddipatla Naga Venkata Sai Krishna, Asit Kumar Sahoo

Affiliations: Both authors are Independent Researchers, San Francisco, CA, USA. Corresponding author: Maddipatla Naga Venkata Sai Krishna; mnvsk97@gmail.com.

Abstract

Business processes and agentic systems contain decisions with different computational requirements. Some require interpretation, contextual judgment, novelty handling, or exception resolution; others are stable enough to execute deterministically. When these boundaries are not known in advance, a natural-language agent provides an expressive starting point, but repeatedly routing stable work through a language model creates avoidable model calls and token consumption. We present Path to Least Non Determinism (PLaND), an evaluation-driven methodology for progressively reducing model-mediated computation within accuracy limits set before evaluation. PLaND begins with an entirely English standard operating procedure (SOP). Its evolver skill instructs a host reasoning agent to inspect development examples and execution records, then propose one revised SOP, called a candidate. A candidate may add Python or Bash code and retain model reasoning for unresolved inputs. Evaluation compares predicted and expected answers on separate examples. This study evaluates these packages, not candidate discovery across independent runs. Using Gemini 3.5 Flash Lite, we tested PLaND on three classification tasks: LEDGAR legal clauses, CFPB consumer complaints, and SpamAssassin email. For each dataset, we used 500 cases for development and 1,000 for selection. We reserved another 500 cases for the final test. At every stage a dataset reached, we ran the comparison three times and report the mean. LEDGAR passed all three final-test runs. Mean accuracy improved from 94.93% to 95.80%, while mean token use fell by 74.56%. SpamAssassin also passed all three final-test runs. Mean accuracy changed slightly from 97.67% to 97.20%, while mean token use fell by 28.13%. CFPB did not pass selection. Its mean accuracy changed from 79.73% to 79.27%, and the hybrid result stayed below the required 80% in all three runs. We therefore did not open the CFPB final test.

Keywords: agentic workflows, agent skills, SOP evolution, deterministic execution, language-model agents, token efficiency, hybrid systems

1 Introduction

Business processes often contain both stable and ambiguous decisions. A language model can interpret unfamiliar inputs and handle exceptions, but it may also spend tokens repeatedly applying the same recognizable rule. For example, a contract clause that explicitly specifies its governing law may be easier to classify than a clause that combines several legal functions. The practical question is: Which decisions can be handled by code while keeping the workflow's quality within an acceptable range?

Existing infrastructure provides ways to package and execute these workflows. The Agent Skills specification organizes reusable instructions in a `SKILL.md` file with optional supporting files [1]. DeepAgents supports loading skills [2], while LangGraph provides graph-based workflow orchestration [3]. These are existing systems on which a methodology can build, not components introduced or independently benchmarked in this paper.

Related research optimizes model programs, prompts, workflows, and reusable skills. DSPy optimizes language-model pipelines against task metrics [4]. AutoFlow and AFlow generate and improve workflows [5, 6], while Automated Design of Agentic Systems searches over agent designs [7]. SkillOpt, SkillRevise, SkillReducer, and ACES study the optimization or evaluation of reusable skills [8-11]. PLaND focuses on a specific change within a workflow: replacing suitable model-mediated decisions with explicit computation, while retaining model reasoning for the remaining inputs and evaluating the complete result.

PLaND begins with an English standard operating procedure (SOP). We call this initial workflow the baseline: the language model follows the English

instructions for every case. A hybrid SOP adds an executable step, meaning code that performs a defined operation when its conditions are met. In our experiments, this step is a Python classification script; cases it cannot classify go to the same language model. We call that path fallback. Figure 1 illustrates these two workflows. Further changes may organize stable steps as nodes in a workflow graph. That is a proposed direction. In this paper, reducing non-determinism means reducing dependence on model-mediated execution. It does not mean finding a globally optimal workflow or proving that repeated model outputs become less variable.

We evaluate this approach on LEDGAR, CFPB, and SpamAssassin. These tasks let us test whether code can avoid model calls, whether accuracy remains within a stated tolerance, and whether the same acceptance decision holds across repeated runs. They are individual classification tasks, not complete business processes.

2 Material and Methods

2.1 Skill structure and executable steps

A skill is a directory whose main instructions are written in `SKILL.md`. Optional references/ files provide supporting instructions or domain material, `scripts/` contains executable code, and `assets/` holds reusable resources such as document templates, images, or lookup tables [1]. A reference file in this directory is supporting material for the agent; it is not a bibliographic citation and does not make a decision deterministic by itself.

An executable step may use Python or Bash and must have defined inputs and outputs. The workflow must actually execute the code and use its result. For example, a Python classification script can return a label directly.

Merely mentioning that script in the English instructions does not establish that it ran.

Figure 1 illustrates the progression using LEDGAR. The English SOP asks the model to read a clause, identify its legal function, compare the allowed labels, and choose one. The hybrid SOP adds a rule-based classifier. Inputs that the rules cannot resolve return to the model. Section 3.2 explains how the benchmark executes this choice.

The allowed labels come from each benchmark and are given to both workflows. The expected label for each case is hidden during execution and used only by the scorer.

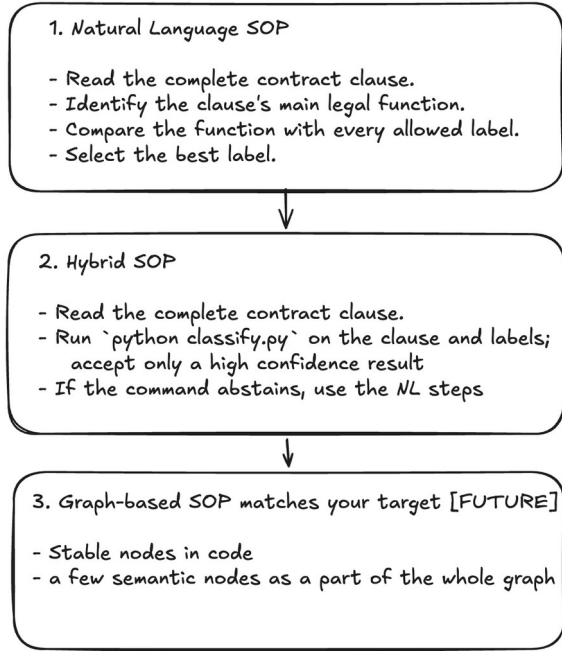


Figure 1. From English instructions to selective code execution. The first two stages summarize the evaluated LEDGAR SOPs. “High confidence” refers to development evidence, not a numerical confidence score. Production logs may reveal recurring paths that can be encoded as steps in a workflow graph, using existing orchestration tools [3]. The third stage illustrates this future direction; it was not evaluated here.

2.2 How an SOP evolves

PLaND uses two skills [12]. The generate-initial-version skill creates an initial agent with one English SOP from the task requirements, approved data sources, and evaluation specification. The pland-evolver skill then instructs a host reasoning agent to run that SOP, inspect its execution records, and propose changes. The two skills are reusable across tasks; the generated SOP and its code are specific to the task. Appendix D summarizes their instructions and links to the complete skill files.

A candidate is a proposed revised version of the whole SOP package: its instructions and any supporting code. Each candidate contains one bounded change, such as adding a Python classification rule. Search means proposing and evaluating candidates until one meets the requirements or the attempt limit is reached. Evolution is

the resulting progression from the current SOP to an accepted revision. A dataset row is one input on which a candidate runs; it is not itself a candidate.

Development, selection, and test are three separate groups of examples from a dataset. Development examples are available while creating and revising the SOP. Selection examples check whether a proposed revision works on cases that did not guide it. The test set stays untouched during selection and provides the final assessment afterward. This separation reduces the risk of accepting a rule that only works on examples already studied. Section 3.1 states the sizes and which evaluations were completed.

The evolution process is:

1. Run the English baseline on development examples. Save its answers, errors, model calls, tokens, and execution records. It must reach the development accuracy floor before hybrid work begins.
2. Inspect those records for recurring patterns or avoidable model work. Propose one change and save the resulting candidate SOP package.
3. Check the candidate on development examples. It must reach the accuracy floor, reduce tokens, and pass the execution checks. If it fails, revise it using development evidence within the attempt limit.
4. Freeze the first candidate that passes development and compare it with the baseline on separate selection examples. Accept it only if every requirement in Section 3.3 passes. Selection rejection ends that dataset’s study; selection answers cannot guide another revision.
5. After selection acceptance, open the reserved test set. Run the final assessment without further tuning. Repeated runs of an unchanged package measure repeatability, not another step of evolution.

Development is a readiness check, not final acceptance. The baseline had to reach 80% accuracy. A hybrid had to reach the same floor, reduce tokens by at least 5%, produce no execution errors, and preserve the frozen inputs and fallback SOP. Its first assessment and all three development repeats had to pass. The plan allowed up to ten baseline attempts and ten hybrid attempts. Reaching the limit without a passing version ended the search. Selection and final testing used the separate requirements in Section 3.3.

Consider this illustrative LEDGAR clause: “This Agreement shall be governed by the laws of the State of California.” Its expected label is Governing Laws. The baseline asks the model to read the clause and choose a label. Suppose development records show that phrases such as “governed by the laws of” recur in this category. A proposed candidate adds a Python rule for that pattern and uses the model when no rule matches. The revised instructions and Python rule together form one candidate.

Success on this one clause is insufficient. The candidate and baseline must process many separate selection clauses, and their predicted labels are compared with the expected answers. A narrower phrase rule would constitute another candidate. This example explains how a proposal could arise; it is not a reconstruction of the

recorded construction process. Appendix D summarizes the saved development attempts.

The runner is the program that executes an SOP on each example. The scorer checks the result. For all three classification tasks here, the scorer gives one point when the returned label exactly matches the expected label and zero otherwise. Accuracy is the proportion of correct labels. Expected answers are used for scoring and are not included in the inputs sent to the model or Python classification script. Section 3.2 explains execution, and Appendix B gives the implementation details.

Figure 2 shows what stays fixed during a comparison. Only the SOP package and its directly related files may change. The task, model, system prompt, selected data, scorer, runtime settings, permissions, and acceptance requirements stay the same for baseline and candidate. File fingerprints and case identifiers check this consistency. Appendix D gives the attempt limits and development checks.

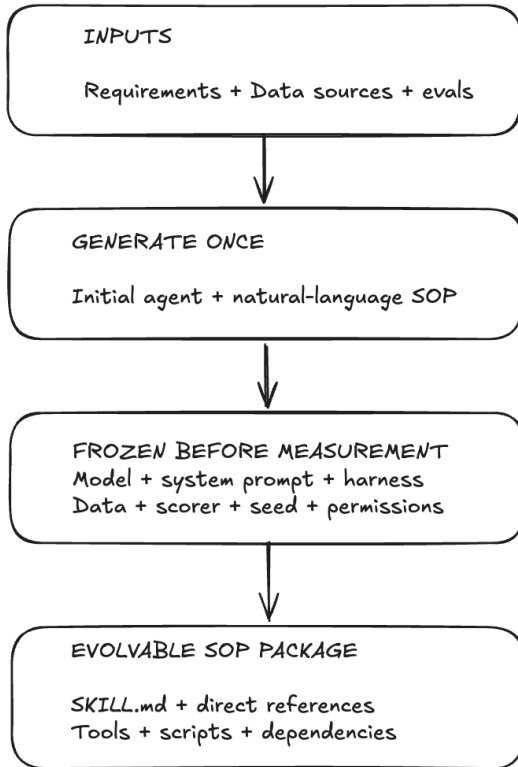


Figure 2. What can change during a comparison. Generate the initial agent and English SOP once, then fix the evaluation setup. Candidate changes are confined to the SOP package. Development records can guide revisions; selection and test answers remain with the evaluator.

2.3 Quality and expense

The quality requirements specify how accurate a workflow must be before lower model use counts as an improvement. For these experiments, both the baseline and candidate must classify at least 80% of cases correctly, and the candidate may lose at most two percentage points of accuracy after accounting for sampling uncertainty. For example, a change from 90% to

88% is a two-point decrease; it is not a 2% relative decrease. Section 3.3 states the four acceptance requirements, including a minimum 5% token reduction.

Let W denote the baseline workflow and W' a proposed replacement. $Q(W)$ is classification accuracy, $E(W)$ is total model input and output tokens, Q_{min} is the minimum acceptable accuracy, and ϵ is the largest allowed accuracy decrease. Here $Q_{min} = 0.80$ and $\epsilon = 0.02$.

The intended comparison is:

$$\begin{aligned}
 E(W') &< E(W) \\
 Q(W) &\geq Q_{min}, \quad Q(W') \geq Q_{min} \\
 Q(W') - Q(W) &\geq -\epsilon
 \end{aligned}$$

These values are study-specific engineering choices recorded in the comparison configuration [12]. They are not thresholds established by the dataset creators or universal standards for these tasks. The records do not provide an application-specific error-cost analysis that would justify 80% or two points for deployment. We therefore interpret acceptance only under these stated tolerances. A real application should choose its accuracy requirements before evaluation, based on the consequences of its errors. Non-inferiority testing provides a framework for assessing an allowed loss [13]; it does not supply the numerical margin used here.

3 Experimental Setup

3.1 Datasets and data splits

We use three public sources: LEDGAR contract clauses through the LexGLUE task [14, 15], consumer complaint narratives from the Consumer Financial Protection Bureau (CFPB) [16], and the SpamAssassin public email corpus [17]. The tasks are to identify a legal clause type, a complaint’s product category, or whether an email is spam. LEDGAR and CFPB each use ten labels; SpamAssassin uses two. Appendix C lists the labels and preparation details.

Table 1. Prepared dataset splits (numbers of examples)

Dataset	Dev.	Selection	Final test
LEDGAR	500	1,000	500
CFPB	500	1,000	500
SpamAssassin	500	1,000	500

In Table 1, Dev. means development. Each dataset has three non-overlapping groups, with the roles defined in Section 2.2: development guides revisions, selection selects the candidate, and the reserved test measures the selected version. The table shows examples prepared, not the number of completed model evaluations. These subsets contain equal numbers per label and do not represent natural label frequencies in production. The dataset manifests and audit records identify the selected examples and check for duplicate identifiers, duplicate content, and overlap with earlier development material [12].

These sizes were fixed before evaluation. The 500 development cases supported candidate creation, the

larger 1,000-case selection set was used for the acceptance decision, and the last 500 stayed untouched for final confirmation. These were study design choices, not statistically optimal sample sizes.

In the completed evaluations reported below, LEDGAR and SpamAssassin passed selection and proceeded to their 500-case final tests. CFPB failed selection, so its 500-case test remained unused. Its reported result therefore covers 1,000 selection examples. The different reported sample sizes follow from this stopping rule; all three datasets had the same split sizes prepared.

3.2 How the baseline and hybrid classify each case

The baseline and hybrid process the same examples. As defined in Section 1, the baseline sends every case to the language model with the English SOP and the list of possible labels. For the illustrative governing-law clause in Section 2.2, the model reads the clause and chooses Governing Laws.

The hybrid tries the Python classification script first. It uses the first matching rule to return a category without calling the model. When no rule matches or the script returns an invalid answer, the workflow sends the same case to the model. This is fallback. For example, a clear governing-law phrase can be handled by code. The email script uses the same principle, with rules for both spam and legitimate email. Rule order matters: the script does not check every possible category conflict before choosing.

The scorer checks every final label against the expected answer, regardless of which path produced it. A matching label scores one and any other answer scores zero. Model-token use is the sum of the input and output tokens reported by the model across all cases. A case answered entirely by the Python script uses no model tokens. Appendix B describes the code calls, output checks, and recorded fields.

The evaluated hybrids preserve the complete English baseline on the fallback path. The results therefore measure the addition of code without shortening the model instructions. The token records show that almost all savings came from avoiding model calls; Appendix B separates the savings by path.

3.3 Acceptance criteria and uncertainty

We fixed four acceptance requirements in the study protocol before the reported evaluations. The same requirements apply to all three datasets [12]:

1. Minimum accuracy: both workflows must classify at least 80% of examples correctly.
2. Accuracy safety check: the lower end of the paired 95% confidence interval for the candidate's accuracy change must be at least -2 percentage points.
3. Token saving: the candidate must use at least 5% fewer total model tokens.
4. Token safety check: the lower end of the paired 95% confidence interval for token reduction must be above zero.

All four must pass, and every evaluated output must be complete and free of execution errors.

A confidence interval accounts for the fact that a different sample of examples could give a different result. Because both workflows process the same cases, the calculation resamples their results as matched pairs. The accuracy check asks whether the less favorable end still lies within the two-point allowance. Appendix A reports each run and its confidence intervals. Appendix B explains the 5,000 resamples used in the calculation.

Figure 3 shows where the acceptance decision belongs in the evolution process described in Section 2.2. The same requirements assess the final test, and a final-test failure would be reported without further tuning.

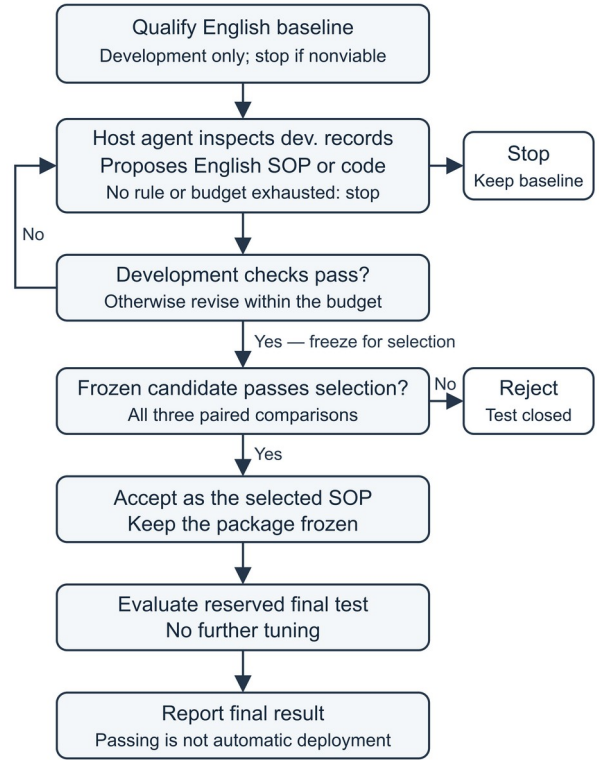


Figure 3. How a candidate is proposed and checked. A candidate is one revised SOP package. Only development examples guide revisions. Separate selection examples decide acceptance; rejection ends the study. After acceptance, the selected candidate is frozen for its reserved test.

3.4 Execution environment and repeated runs

The experiments used Gemini 3.5 Flash Lite through a hosted service. Appendix B records the model settings and implementation details.

Three paired executions ran on every reached split: development, selection, and, where permitted, final test. One paired run means that both workflows process the same evaluation examples under matching model and runtime settings. Repeating an evaluation does not create new cases or new candidate SOPs.

Rate limits were retried, and interrupted runs resumed from saved receipts rather than counting these events as case failures. Two blocked SpamAssassin selection emails were replaced with unique same-label cases under recorded amendments, without reducing the dataset size. Appendix C gives the replacement details.

4 Results

Table 2 presents the main results. All accuracy and token comparisons show the natural-language baseline first and the hybrid second. Headline accuracy and token reduction are means across three runs, while every repeat still had to pass its gate. A pass means that the requirements in Section 3.3 were met; it does not mean that accuracy was identical.

Table 2. Main evaluation results

Dataset and split	Baseline to hybrid result
LEDGAR, final-test, 500 cases	Mean accuracy: 94.93% → 95.80%; mean token reduction: 74.56%; Pass
CFPB, selection, 1,000 cases	Mean accuracy: 79.73% → 79.27%; mean token reduction: 23.45%; Reject
SpamAssassin, final-test, 500 cases	Mean accuracy: 97.67% → 97.20%; mean token reduction: 28.13%; Pass

4.1 LEDGAR

On the 500 LEDGAR final-test clauses, mean accuracy changed from 94.93% for the baseline to 95.80% for the hybrid. The quality and token requirements passed in every repeat.

The hybrid handled 370 clauses through the Python classification script and sent the remaining 130 to the model. Model calls therefore fell from 500 to 130 per run, and the mean token reduction was 74.56%.

4.2 CFPB

CFPB's hybrid reduced model calls from 1,000 to 788 per selection run, with a mean token reduction of 23.45%, but mean accuracy changed from 79.73% to 79.27%. Every hybrid repeat missed the 80% minimum. The candidate was rejected and the reserved test was not evaluated. Lower token use did not compensate for failing the accuracy floor.

Further development of the complaint SOP and its Python rules might improve the result. Additional tailoring is outside this comparison, so these results do not test that possibility. We evaluate the saved packages under common requirements to keep the assessment consistent across tasks. This does not establish equal development effort or the best achievable accuracy for each task.

4.3 SpamAssassin

SpamAssassin's hybrid reduced model calls from 500 to 323 per final-test run. Mean accuracy changed from 97.67% to 97.20%, and the mean token reduction was 28.13%. It passed the quality and token requirements in every repeat. Its small accuracy loss stayed within the allowed tolerance; passing does not mean that accuracy improved.

5 Discussion

5.1 Model reasoning as a resource

The main systems argument is that non-deterministic reasoning should be allocated where it is needed rather than applied uniformly across a workflow. Business

processes often contain both stable and ambiguous decisions. When stable regions are repeatedly routed through a language model, the workflow spends model calls and tokens on work that ordinary computation may be able to execute directly.

PLaND provides a controlled way to move that boundary. The English baseline offers a flexible starting point when the structure of the task is not fully known. Development runs reveal errors, repeated reasoning, and expense. The evolver skill guides a host agent to propose a bounded change. Evaluation then determines whether the resulting workflow meets the chosen requirements.

The useful unit of change is not necessarily an entire workflow step. LEDGAR shows that some inputs to a semantic classification task can be handled by explicit rules, while the remaining inputs still require the model. Contract-clause classification remains a semantic problem overall. Its more recognizable cases nevertheless offer opportunities to avoid model calls, beyond mechanical operations such as counting or schema validation.

The benefit must be considered alongside the error tolerance. LEDGAR reduced tokens with slightly higher mean accuracy. SpamAssassin reduced tokens with a small accuracy loss within the allowed tolerance. CFPB did not satisfy the same evaluation criteria. The three results support evaluating each proposed substitution rather than assuming that fewer model calls imply an acceptable workflow.

5.2 What token savings measure

PLaND's generation instructions ask the agent to consider resource use beyond tokens, including CPU, memory, storage, network access, caching, and setup work. The framework can represent different expense objectives. In the experiments reported here, however, the acceptance decision used model tokens as its only efficiency measure. Model-call counts explain the mechanism; they are not a second independently optimized objective.

Tokens and calls are useful measures of model use, but they do not directly establish dollar, energy, or end-to-end latency savings. Inference time can depend on the service, hardware utilization, and concurrency. Code also has execution and maintenance costs. Claims about those costs require corresponding measurements and are outside the present evaluation.

5.3 From classification tasks to business workflows

The retained tasks resemble individual decisions within business processes. They do not include long-running state, user interaction, external side effects, or recovery from partial failure. Extending the result to a complete workflow would require evaluating those behaviors as well as label accuracy. The final-test evidence is limited to LEDGAR and SpamAssassin with one hosted model. The email corpus is old, and these balanced subsets do not establish performance on current production data. Replacing blocked emails may also bias selection toward content the provider can process.

The small number of repeated runs limits conclusions about variability. The experiments evaluate the selected packages, not how reliably an agent can discover useful rules across independent attempts. They also do not compare PLaND with a trained small classifier.

6 Future Work

The next step is to repeat the full evolution process from scratch. Each independent run would start from the same baseline and development data, then let the host agent inspect records, propose code, and evaluate candidates under fixed rules. This would measure how often the evolver finds a useful candidate and how much development work it requires. The present study records the development path used for each dataset and evaluates the resulting packages. It does not measure discovery reliability across independent starts.

A second direction is to evaluate complete workflows with state, tool calls, and recoverable failures. The scorer would need to check whether the task was completed correctly, not just whether a label matched. Such experiments would test whether local token savings remain useful when deterministic steps interact with a wider process.

A third direction is to study maintenance over time. Input distributions can change, making a previously acceptable rule unreliable. A longitudinal evaluation could test monitoring, rollback, and returning affected inputs to the model. These mechanisms are proposed future work, not features validated by the present experiments.

Runtime evidence could also guide automatic construction of a more deterministic workflow. Production logs may reveal recurring input patterns, repeated tool sequences, failures, and expensive model calls. A future system could use those records to propose a graph in which stable paths execute as code and ambiguous cases retain model reasoning, as illustrated in Figure 1. Such proposals would still need evaluation on separate examples before deployment. Whether an agent can reliably generate and maintain this complete workflow remains an open question.

Further comparisons should include other models, languages, and methods for generating code from examples or execution records. Comparisons with trained small classifiers would help show when selective rules are useful. Application-specific error costs and direct resource measurements would make acceptance decisions more relevant to deployment.

7 Conclusion

PLaND is a methodology for reducing model-mediated work through evaluated changes to an English SOP. Its central mechanism is selective code execution with model fallback. On LEDGAR, the evaluated hybrid had a mean token reduction of 74.56%, with mean accuracy changing from 94.93% to 95.80%. SpamAssassin had a mean token reduction of 28.13%, with mean accuracy changing from 97.67% to 97.20%. Both passed the stated requirements in all three paired runs. CFPB did not pass selection, and its reserved test was not evaluated. These results show why both quality

and token use should be tested before accepting a substitution. The study evaluates the selected packages; it does not establish how reliably independent evolver runs will discover useful candidates or how PLaND performs in complete production workflows.

Acknowledgements and Disclosures

The authors used AI-assisted tools for coding, experiment support, and manuscript preparation. The authors are responsible for the reported results and the final text. No external funding was received, and the authors declare no competing interests.

Data and Code Availability

The [PLaND repository](#) provides the two methodology skills, shared classification runner and scorer, saved SOPs, and evaluation records [12]. Appendix B identifies the current evidence and gives the verification commands.

Saved run records include case identifiers, expected and predicted labels, correctness, the path used, model calls, tokens, timing, and file fingerprints. Raw datasets are subject to their source terms and are not included in the repository. Exact regeneration requires the recorded input files and model version. In particular, the CFPB inputs came from a saved local API snapshot that is not redistributed. A fresh download from the live database would constitute a different input snapshot. Repository verification checks saved files and code tests; it does not rerun model inference.

Appendix A Individual held-out executions

Repeats 1, 2, and 3 correspond to identifiers 20260903, 20260904, and 20260905. Each pair uses identical cases. Repeats are not additional independent samples and are not pooled to enlarge the case count.

Table A1 shows what happened in each run. The baseline and hybrid values are shown in that order, separated by a slash.

Table A1. Results from each run

Dataset and stage	Run	Accuracy: baseline / hybrid (%)	Model tokens: baseline / hybrid
LEDGAR selection	1	97.0 / 96.7	472,670 / 140,424
LEDGAR selection	2	97.3 / 96.9	472,717 / 140,392
LEDGAR selection	3	97.4 / 96.9	472,800 / 140,484
LEDGAR final test	1	94.8 / 95.8	242,557 / 61,653
LEDGAR final test	2	95.0 / 95.8	242,421 / 61,700
LEDGAR final test	3	95.0 / 95.8	242,372 / 61,707
CFPB selection	1	80.0 / 79.4	700,508 / 536,424
CFPB selection	2	79.6 / 78.9	700,526 / 536,141
CFPB selection	3	79.6 / 79.5	700,470 / 536,111
SpamAssassin selection	1	99.0 / 97.9	2,636,096 / 1,788,601
SpamAssassin selection	2	98.8 / 98.1	2,636,096 / 1,788,658
SpamAssassin selection	3	99.0 / 98.1	2,636,117 / 1,788,728
SpamAssassin final test	1	97.8 / 97.2	1,464,817 / 1,052,802
SpamAssassin final test	2	97.4 / 97.0	1,465,131 / 1,052,891
SpamAssassin final test	3	97.8 / 97.4	1,464,835 / 1,052,948

Table A2 shows the uncertainty check for the same runs. CI means confidence interval. It is not the spread across the three runs.

Table A2. Confidence intervals used for acceptance

Dataset and stage	Run	Accuracy difference 95% CI (points)	Token reduction 95% CI (%)
LEDGAR selection	1	-1.10 to 0.50	67.38 to 73.16
LEDGAR selection	2	-1.20 to 0.40	67.40 to 73.17
LEDGAR selection	3	-1.30 to 0.30	67.39 to 73.15
LEDGAR final test	1	-0.40 to 2.60	70.65 to 78.36
LEDGAR final test	2	-0.60 to 2.20	70.62 to 78.34
LEDGAR final test	3	-0.60 to 2.20	70.61 to 78.33
CFPB selection	1	-1.60 to 0.40	20.56 to 26.33
CFPB selection	2	-1.80 to 0.40	20.60 to 26.38
CFPB selection	3	-1.20 to 1.10	20.60 to 26.38
SpamAssassin selection	1	-1.80 to -0.50	28.01 to 36.63
SpamAssassin selection	2	-1.40 to -0.10	28.01 to 36.63
SpamAssassin selection	3	-1.60 to -0.30	28.00 to 36.63
SpamAssassin final test	1	-1.60 to 0.40	22.13 to 35.18
SpamAssassin final test	2	-1.60 to 0.60	22.14 to 35.18
SpamAssassin final test	3	-1.40 to 0.60	22.13 to 35.17

For accuracy, positive values favor the hybrid and negative values favor the baseline. The lower number had to be at least -2.00 points. For token reduction, both numbers had to be above zero. CFPB passed these interval checks but was rejected because hybrid accuracy was below 80% in all three runs. Calculations use unrounded values.

Example: LEDGAR final-test run 1 has an accuracy interval of -0.40 to 2.60 points. Because -0.40 is above the -2.00-point limit, it passed the accuracy check. Its token-reduction interval, 70.65% to 78.36%, was above zero, so it passed the token check.

Table A3. Selection means and decisions

Dataset	Mean accuracy B → H (%)	Mean token reduction (%)	Decision
LEDGAR	97.23 → 96.83	70.29	Accept
CFPB	79.73 → 79.27	23.45	Reject
SpamAssassin	98.93 → 98.03	32.15	Accept

Table A4. Final-test means and model calls

Dataset	Mean accuracy B → H (%)	Calls B → H	Mean token reduction (%)
LEDGAR	94.93 → 95.80	500 → 130	74.56
SpamAssassin	97.67 → 97.20	500 → 323	28.13

Appendix B Reproduction details

B.1 Runtime and scoring

The evaluated model was google/gemini-3.5-flash-lite, accessed through OpenRouter with Google AI Studio as the sole permitted provider endpoint. The setup used DeepAgents 0.7.12, low reasoning effort, a 1,024-token completion limit, non-streaming requests, and a 300-second timeout. Temperature was omitted, leaving the service default. Provider fallback was disabled. The configuration hash identifies request and routing settings, not model weights. Hosted weight bytes and a stable system fingerprint were unavailable.

The runner used eight concurrent case workers, preserving the frozen setting between arms. HTTP 429 responses were recorded and retried, honoring Retry-After or using exponential backoff with case-specific jitter and a maximum delay of 300 seconds. They were not classified as failed examples. Interrupted runs retained completed case receipts and resumed missing work after checking frozen inputs and configuration.

Three paired executions ran on every reached split with the same cases, SOPs, label format, and runtime. Their identifiers are 20260903, 20260904, and 20260905. Although the plan calls them repeat seeds, the adapter records `seed_supported: false` and sends no inference seed. They are repeat identifiers, not controlled model-randomness settings. Data sampling and bootstrap sampling used seed 20260902; the token bootstrap used that value plus one. We report each held-out result without assuming deterministic service behavior.

The shared runner calls the Python classifier directly; it does not ask the model to launch a shell for each case. Both arms request label-only JSON. The scorer compares the decoded label with the expected label by exact string equality, without changing capitalization, punctuation, or aliases. Expected answers are not supplied to either classifier. Invalid answers score zero; execution errors are recorded separately.

Completion tokens include any reported reasoning tokens, which are not counted twice. Cached input tokens remain part of input-token use. The reported totals cover completed evaluation runs, not construction work, screening, failed attempts, or retries. Resumed-run wall times are not complete execution times and are not headline results.

B.2 Uncertainty calculation

The comparison code sorts records by case identifier and draws 5,000 paired bootstrap resamples [18]. Each resample selects cases with replacement and retains both

workflows' results for each selected case. It then recalculates the accuracy difference and token reduction. The 2.5th and 97.5th percentiles form the reported 95% interval. For acceptance, the lower endpoint must be at least -2 percentage points for accuracy change and strictly above zero for token reduction. It is not a guaranteed worst possible outcome.

These intervals describe uncertainty from sampling the prepared examples, not changes in the model or production data. The calculation treats cases as exchangeable; related clauses or emails may violate that assumption. Repeats are not pooled as independent samples.

B.3 Token savings by path

Fallback input-token counts matched the baseline on every corresponding final-test case. In LEDGAR final-test repeat 1, the hybrid saved 180,904 tokens: 180,828 from avoiding model calls and 76 from shorter model outputs on fallback cases. In SpamAssassin final-test repeat 1, the corresponding amounts were 412,015, 411,938, and 77 tokens. The measured savings therefore came almost entirely from bypassing model calls. They did not come from shortening the fallback prompt. For one SpamAssassin selection email, provider-reported input counts differed by 1, 3, 2 tokens across the three pairs despite the unchanged prompt-construction contract. The saved receipts do not explain this small discrepancy; it is retained in the reported totals.

B.4 Evidence and verification

The collection root is `experiments/gemini-3.5-flash-lite/`. It contains the frozen plan, recorded amendments, controllers, comparisons, and evidence manifests. `paper-calculations.json` lists the exact files and hashes used for every reported result, including the superseded SpamAssassin records. Distinct conditions are retained separately.

The manuscript audit is in `2026-09-06-manuscript/`. Its `paper-calculations.json` identifies every reported pair, file hash, SOP hash, classifier hash, gate, and recomputed interval. `evidence-files.json` inventories saved collection and code files; `paper-artifacts.json` identifies manuscript outputs. The provenance addendum records superseded metadata issues without changing frozen results.

Run `uv sync --project reproduce --frozen, then reproduce/.venv/bin/python reproduce/verify.py.` Run `reproduce/.venv/bin/python paper/audit_paper.py --artifacts` to recompute results and verify manuscript files. These commands are offline and do not open CFPB's reserved test. The audit derives accuracy and tokens from case outputs, independently rebuilds paired bootstrap intervals, and checks saved comparisons. Command ledgers retain original collection and retry commands.

Appendix C Labels and sources

LEDGAR uses Governing Laws, Counterparts, Notices, Entire Agreements, Severability, Amendments, Survival, Assignments, Expenses, and Terms. These are

corpus categories [14] distributed through LexGLUE [15]. All three study splits were newly sampled from the source training partition after prior-case exclusions.

CFPB uses the product field associated with published narratives in the saved snapshot [16]. The labels are Mortgage; Checking or savings account; Student loan; Money transfer, virtual currency, or money service; Vehicle loan or lease; Prepaid card; Payday loan, title loan, personal loan, or advance loan; Credit card; Debt collection; and Credit reporting or other personal consumer reports. These strings come from the frozen snapshot, not label discovery.

SpamAssassin uses ham from easy-ham and hard-ham archives and spam from spam and spam-2 archives dated 20030228 [17]. In its final selection set, blocked spam messages mail-3125dcdc7726abc1c128 and mail-449598896e25273135c3 were replaced by mail-023db858450b084e7238 and mail-0ae0fc25ca50549e73a8, respectively. The set retained 500 ham and 500 spam cases. Data audits record source hashes, case identifiers, label balance, and duplicate/overlap checks.

Each amendment restarted all three selection pairs on the amended set. Development and final-test inputs, packages, model settings, and thresholds stayed unchanged. Original blocked attempts and superseded outputs were retained. Screening checked whether the provider would process the email, not whether it classified it correctly. The responses were not used to tune rules, and provider safeguards were not weakened.

Appendix D What the two PLaND skills instruct

The [generate-initial-version skill](#) specifies the initial agent, one English SOP, approved data access, and task-specific runner and scorer files. It derives the scaffold from requirements and the evaluation structure without copying case answers into the agent. Python or Bash replacements for SOP steps are introduced only during subsequent evolution.

The [pland-evolver skill](#) instructs the host agent to measure the current SOP, inspect development records, propose one bounded change, and compare the candidate under fixed conditions. It saves the hypothesis, package changes, measurements, and accept/reject decision. A rejected candidate leaves the previous accepted version in place. Once a reserved test result is opened, the skill instructs the agent to stop evolving and report it.

The approved study plan allowed up to ten English-baseline attempts and ten hybrid attempts. Exhausting the budget was not acceptance. The baseline first had to reach 80% development accuracy. A hybrid then had to reach that floor, reduce tokens by at least 5%, produce no execution errors, and satisfy the frozen-input and SOP contracts. Its primary assessment and all three development repeats had to pass before selection. These development checks used observed results; the bootstrap requirements applied to selection and final testing.

Every baseline qualified on its first attempt. LEDGAR and CFPB qualified their second hybrid candidate;

SpamAssassin qualified its first. Table D1 reports the development repeats.

Table D1. Development attempts and mean accuracy

Dataset	B attempts	H attempts	Mean accuracy B → H (%)
LEDGAR	1	2	96.53 → 97.20
CFPB	1	2	81.67 → 81.87
SpamAssassin	1	1	98.87 → 98.67

The evolver requires a new executable step to preserve the complete English fallback, with any later instruction shortening evaluated separately. The evaluated hybrids followed that requirement.

References

- [1] Agent Skills. Specification. [Official documentation](#). Accessed September 6, 2026.
- [2] LangChain. Skills in DeepAgents. [Official documentation](#). Accessed September 6, 2026.
- [3] LangChain. LangGraph overview. [Official documentation](#). Accessed September 6, 2026.
- [4] Khattab O, et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. 2023. [arXiv:2310.03714](#).
- [5] Li Z, et al. AutoFlow: Automated Workflow Generation for Large Language Model Agents. 2024. [arXiv:2407.12821](#).
- [6] Zhang J, et al. AFlow: Automating Agentic Workflow Generation. 2024. [arXiv:2410.10762](#).
- [7] Hu S, Lu C, Clune J. Automated Design of Agentic Systems. 2024. [arXiv:2408.08435](#).
- [8] Yang Y, et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. 2026. [arXiv:2605.23904](#).
- [9] Liu Y, et al. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. 2026. [arXiv:2606.01139](#).
- [10] Gao Y, et al. SkillReducer: Optimizing LLM Agent Skills for Token Efficiency. 2026. [arXiv:2603.29919](#).
- [11] Kevin C, et al. Evaluating Skills, Not Just Agents: Agentic Continuous Evaluation of Skills. 2026. [arXiv:2608.20614](#).
- [12] Maddipatla Naga Venkata Sai Krishna, Asit Kumar Sahoo. PLaND experiment records and implementation. Companion research artifact. 2026. [Public repository](#). [Frozen collection](#). Exact plan, evidence, and audit paths are given in Appendix B. Author-produced provenance, not independent validation.
- [13] Walker E, Nowacki AS. Understanding Equivalence and Noninferiority Testing. Journal of General Internal Medicine. 2011;26(2):192–196. [doi:10.1007/s11606-010-1513-8](#).
- [14] Tugener D, von Däniken P, Peetz T, Cieliebak M. LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts. Proceedings of LREC. 2020:1235–1241. [ACL Anthology](#).
- [15] Chalkidis I, et al. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. Proceedings of ACL. 2022:4310–4330. [ACL Anthology](#).
- [16] Consumer Financial Protection Bureau. Consumer Complaint Database. [Official dataset](#). Accessed September 6, 2026.
- [17] Apache SpamAssassin. Public mail corpus. [Official dataset](#). Accessed September 6, 2026.
- [18] Efron B. Bootstrap Methods: Another Look at the Jackknife. The Annals of Statistics. 1979;7(1):1–26. [doi:10.1214/aos/1176344552](#).